

■ Load Shedding Smart Scheduler

Complete Beginner's Build Guide — South Africa Edition

WhatsApp · Gemini AI · EskomSePush · Open-Meteo · FastAPI · Railway

College Project | 100% Free Stack | Zero Hardware

■ 0

Total Cost

6 Weeks

Build Time

4 Real APIs

Live Data Sources

1 Working App

WhatsApp + AI

Table of Contents

01	What Are We Building?
02	Final Tech Stack
03	How Everything Connects — The Workflow
04	Folder Structure
05	Week-by-Week Build Plan
06	Step-by-Step: What to Do First
07	Free API Setup Guide
08	Deployment on Railway
09	Your Demo Script

01 — What Are We Building?

A **WhatsApp-native AI energy advisor** for South African households. Every morning at 6 AM, your app automatically pulls the real load shedding schedule, checks the solar forecast, looks at the household's simulated battery state, and sends a plain-English daily plan over WhatsApp — telling the family exactly what appliances to run and when.

■ What the family receives on WhatsApp at 6 AM

Good morning! ■■ Today's load shedding: 10 AM–12 PM & 6 PM–8 PM (Stage 3).

■ Battery: 78% | ■■ Solar peak: 11 AM–2 PM (est. 2.1 kWh)

My plan for today:

- Run geyser NOW (grid available, battery healthy)
- Washing machine → 1 PM solar window
- Water pump → avoid during cuts

Reply CHANGE, STATUS, or ask me anything.

What is simulated vs real?

Component	Real or Simulated?	Why
Load shedding schedule	■ REAL	EskomSePush API
Solar / weather forecast	■ REAL	Open-Meteo API
WhatsApp messages	■ REAL	Twilio Sandbox
AI scheduling brain	■ REAL	Gemini API
Inverter + battery data	■ Simulated class	No hardware needed
Household appliances	■ Simulated class	Hardcoded realistic data

02 — Final Tech Stack

Real Data Sources

Tool	What it does	Cost	How to get it
EskomSePush API	Real load shedding schedules for SA	Free — 50 calls/day	Sign up at eskomsepush.com
Open-Meteo API	Real solar irradiance + weather forecast	Free — no key needed	Just a URL, call it directly

Simulated (Your Code)

Tool	What it does	Cost	How to get it
SimulatedInverter	Fake battery %, solar watts, grid status	Free	Python class you write
SimulatedHousehold	Fake appliance list with deadlines + wattages	Free	Python class you write

AI Brain

Tool	What it does	Cost	How to get it
Gemini API	Reasons over all data, produces the daily plan	Free — 1500 req/day	Google AI Studio, no credit card

Backend

Tool	What it does	Cost	How to get it
FastAPI	Python web framework — your app's server	Free / open source	pip install fastapi
APScheduler	Runs the 6 AM job automatically	Free / open source	pip install apscheduler
SQLite	Stores plans + household data	Free, built into Python	No setup needed

Messaging

Tool	What it does	Cost	How to get it
Twilio WhatsApp Sandbox	Sends + receives WhatsApp messages	Free for dev	Sign up at twilio.com

Deployment

Tool	What it does	Cost	How to get it
Railway	Hosts your FastAPI app in the cloud	Free credits (\$5/month)	railway.app

03 — How Everything Connects

Here is the exact sequence of events every single day. Read this slowly — once you understand this flow, everything else will make sense.



6:00 AM — Scheduler Wakes Up

APScheduler (running inside your FastAPI app on Railway) fires a job every day at 6 AM automatically. This is what kicks everything off. You don't press anything — it just happens.



Fetch Real Outage Schedule

Your app calls the EskomSePush API with the household's area name (e.g. 'Cape Town Zone 3'). It gets back today's outage windows — e.g. 10 AM–12 PM, 6 PM–8 PM — with the current stage number.



Fetch Solar Forecast

Your app calls Open-Meteo with the household's latitude and longitude. It gets back hourly solar irradiance values. You convert this to estimated kWh the panels will produce each hour.



Read Simulated Inverter State

Your SimulatedInverter class generates a morning snapshot: battery is at 78%, grid is currently on, solar output is 0W (it's 6 AM). This is a Python class — no hardware needed.



Read Household Profile

Your SimulatedHousehold class returns the appliance list: geyser (3kW, needed by 8 AM), washing machine (2kW, flexible), fridge (always on). Minimum battery reserve: 30%.



Gemini Agent Reasons

All 4 inputs are sent to the Gemini API as a prompt. Gemini reasons: 'Grid is on now, battery is healthy, geyser should run immediately. Outage at 10 AM means washing machine must finish before then or wait for solar window at 1 PM.' It writes a plain-English plan.



Save Plan to SQLite

The generated plan is saved to your local SQLite database with a timestamp. This lets you look up what was sent to any household on any day.



Send WhatsApp Briefing

Your app calls Twilio's API and sends the plan as a WhatsApp message to the household's number. The family wakes up to the plan already in their WhatsApp.

9

User Replies — Gemini Parses

If the user replies ('change washing machine to evening'), Twilio sends that reply to your FastAPI webhook. Gemini reads the reply + the current plan, understands the intent, updates the plan, and sends a confirmation back on WhatsApp.

04 — Folder Structure

Create this exact folder structure on your laptop before writing any code.

```
smart-scheduler/
├── simulators/
│   ├── __init__.py
│   ├── inverter.py          ← SimulatedInverter class
│   ├── household.py        ← SimulatedHousehold class
│   └──
├── integrations/
│   ├── __init__.py
│   ├── eskomsepush.py      ← Calls EskomSePush API
│   ├── openmeteo.py        ← Calls Open-Meteo API
│   ├── whatsapp.py         ← Twilio send / receive
│   └──
├── agent/
│   ├── __init__.py
│   ├── scheduler.py        ← Gemini agent logic
│   └──
├── api/
│   ├── __init__.py
│   ├── main.py             ← FastAPI app + APScheduler
│   └──
├── db/
│   ├── __init__.py
│   ├── models.py           ← SQLite schema
│   └──
├── .env                    ← All your API keys (never share this)
├── requirements.txt         ← Python packages list
└── README.md
```

■ Beginner Tip

Each folder is called a 'module'. The `__init__.py` file inside each folder just tells Python 'treat this folder as a module'. You can leave them empty for now.

requirements.txt — copy this exactly

```
fastapi
uvicorn
apscheduler
requests
python-dotenv
google-generativeai
twilio
sqlalchemy
```

Run `pip install -r requirements.txt` once and you're set.

05 — Week-by-Week Build Plan

WEEK 1 Simulator Classes

- Create folder structure
- Write SimulatedInverter class
- Write SimulatedHousehold class
- Test both classes print realistic data

You'll learn: Python classes, data modelling, realistic randomness

WEEK 2 Real Data Integrations

- Sign up for EskomSePush — get API key
- Write eskomsepush.py — fetch today's schedule
- Write openmeteo.py — fetch solar forecast
- Print both outputs and verify they look right

You'll learn: API calls with requests library, JSON parsing

WEEK 3 Gemini AI Agent

- Sign up for Google AI Studio — get Gemini API key
- Write scheduler.py — the agent prompt
- Feed all 4 inputs to Gemini
- Print the generated plan and tweak the prompt

You'll learn: Prompt engineering, AI API calls, structured output

WEEK 4 WhatsApp Integration

- Sign up for Twilio — set up WhatsApp Sandbox
- Write whatsapp.py — send a test message
- Set up webhook to receive replies
- Wire reply → Gemini → response back

You'll learn: Webhooks, two-way messaging, conversational AI

WEEK 5 FastAPI + Scheduler

- Write main.py with FastAPI app
- Add APScheduler — trigger 6 AM job
- Connect all pieces end-to-end
- Run locally and test full flow

You'll learn: Web frameworks, scheduled jobs, end-to-end integration

WEEK 6 Deploy + Polish

- Push code to GitHub
- Connect Railway to your GitHub repo
- Set environment variables on Railway
- Run a live demo from a real phone

You'll learn: Cloud deployment, environment variables, live demo

06 — Step-by-Step: What to Do First

This section tells you exactly what to do, in what order, technically. Follow this like a recipe.

Step 1 Set up your project

1. Open your terminal / VS Code
2. Run: `mkdir smart-scheduler && cd smart-scheduler`
3. Create all the folders and `__init__.py` files shown in Section 04
4. Run: `python -m venv venv` (creates a virtual environment)
5. Run: `source venv/bin/activate` (Mac/Linux) or `venv\Scripts\activate` (Windows)
6. Create `requirements.txt` with the packages from Section 04
7. Run: `pip install -r requirements.txt`

Step 2 Build SimulatedInverter

1. Open `simulators/inverter.py`
2. Create a class `SimulatedInverter` with an `__init__` that sets `battery=78`, `solar_watts=0`, `grid_on=True`
3. Add a method `get_snapshot()` that returns a dictionary with these values
4. Make battery drain slightly if `grid_on` is `False` (simulate outage)
5. Make `solar_watts` follow a bell curve — 0 at 6 AM, peaks at 1200W at noon, 0 by 6 PM
6. Test it: `python -c 'from simulators.inverter import SimulatedInverter; print(SimulatedInverter().get_snapshot())'`

Step 3 Build SimulatedHousehold

1. Open `simulators/household.py`
2. Create a class `SimulatedHousehold`
3. Add a method `get_appliances()` that returns a list of dicts
4. Each appliance has: `name`, `wattage`, `priority` (1-5), `deadline`, `flexible` (True/False)
5. Example: `{'name': 'Geyser', 'watts': 3000, 'priority': 1, 'deadline': '08:00', 'flexible': False}`
6. Add a method `get_rules()` that returns `min_battery_reserve: 30`, `area: 'Cape Town Zone 3'`

Step 4 Connect EskomSePush

1. Go to eskomsepush.com and sign up — you get a free API key
2. Add `ESKOMSEPush_KEY=your_key` to a `.env` file in the root folder
3. Open `integrations/eskomsepush.py`
4. Use the requests library to call: `https://developer.sepush.co.za/business/2.0/area`
5. Pass your API key in the headers and the area ID as a parameter
6. Parse the JSON response to extract outage start/end times
7. Test it: `python -c 'from integrations.eskomsepush import get_schedule; print(get_schedule())'`

Step 5 Connect Open-Meteo

1. Open `integrations/openmeteo.py`
2. No API key needed — just call the URL directly
3. URL:
`https://api.open-meteo.com/v1/forecast?latitude=-33.9&longitude;=18.4&hourly;=shortwave_radiation`
4. Change latitude/longitude to your test household location
5. Parse the hourly `shortwave_radiation` values — this is solar energy in W/m^2
6. Multiply by 0.0015 (rough panel efficiency) to get estimated kWh per hour
7. Test it the same way as above

Step 6 Build the Gemini Agent

1. Go to aistudio.google.com — sign in with Google, get your free API key
2. Add `GEMINI_KEY=your_key` to your `.env` file
3. Open `agent/scheduler.py`
4. Import `google.generativeai` and configure with your key
5. Write a function `generate_plan(outage, solar, inverter, household)` that builds a prompt string
6. The prompt should explain the situation and ask Gemini to produce a WhatsApp-friendly schedule
7. Call `model.generate_content(prompt)` and return the text
8. Test by calling this function with fake data and printing the output

Step 7 Wire WhatsApp

1. Go to [twilio.com](https://www.twilio.com) — sign up free, go to WhatsApp Sandbox in the console
2. Save TWILIO_SID, TWILIO_TOKEN, TWILIO_FROM to .env
3. Open integrations/whatsapp.py
4. Write a send_message(to, body) function using the Twilio Python library
5. Write a FastAPI POST route at /webhook that Twilio calls when user replies
6. In that route, read the reply text, pass it to Gemini with the current plan, send response back
7. Configure the Twilio Sandbox webhook URL to point to your Railway app URL + /webhook

Step 8 Put it all together in main.py

1. Open api/main.py
2. Create a FastAPI app instance
3. Create a function run_daily_job() that calls all integrations in order (steps 4→7)
4. Add APScheduler: scheduler.add_job(run_daily_job, 'cron', hour=6, minute=0)
5. Start the scheduler when the app starts using a FastAPI lifespan event
6. Add the /webhook POST route from Step 7
7. Run locally: uvicorn api.main:app --reload
8. Check that the scheduler is running and the webhook endpoint responds

07 — Free API Setup Guide

EskomSePush

eskomsepush.com

1. Go to eskomsepush.com
2. Click 'Business API' → Sign up with email
3. Verify your email
4. Go to your dashboard → copy the API token
5. Add to .env: `ESKOMSE_PUSH_KEY=paste_your_token_here`
6. Test: call `/api/status` endpoint to confirm it works

Open-Meteo

open-meteo.com

1. No signup needed at all
2. Just use the URL directly in your code
3. URL format:
`https://api.open-meteo.com/v1/forecast?latitude=LAT&longitude=LON&hourly=shortwave_radiation`
4. Replace LAT and LON with your test location coordinates
5. For Cape Town use: `latitude=-33.9249, longitude=18.4241`
6. No key needed in .env for this one

Google Gemini

aistudio.google.com

1. Go to aistudio.google.com
2. Sign in with your Google account
3. Click 'Get API Key' → Create API Key
4. Copy the key
5. Add to .env: `GEMINI_KEY=paste_your_key_here`
6. Free tier: 1500 requests/day — more than enough

1. Go to twilio.com → Sign up free
2. Go to Console → Messaging → Try it out → Send a WhatsApp message
3. Follow instructions to join the Sandbox (you'll send a code to a Twilio number)
4. Note your Account SID, Auth Token, and Sandbox number
5. Add to .env: `TWILIO_SID=xxx`, `TWILIO_TOKEN=xxx`, `TWILIO_FROM=whatsapp:+14155238886`
6. For webhooks: use Railway URL + /webhook as the 'When a message comes in' URL

■ **Important — .env file**

NEVER push your .env file to GitHub. Add '.env' to your .gitignore file. All keys go as environment variables on Railway — never hardcode them in your Python files.

08 — Deployment on Railway

1	Push your code to GitHub (create a free account if you don't have one)
2	Go to railway.app → Sign up with your GitHub account
3	Click 'New Project' → 'Deploy from GitHub repo' → select your repo
4	Railway auto-detects Python and installs requirements.txt
5	Go to your project → Variables tab → add all your .env keys here
6	Go to Settings → add a Start Command: <code>uvicorn api.main:app --host 0.0.0.0 --port \$PORT</code>
7	Railway gives you a public URL like: <code>https://smart-scheduler.up.railway.app</code>
8	Copy this URL → go to Twilio Sandbox settings → paste URL + <code>/webhook</code> as the incoming message URL
9	Your app is now live and will run the 6 AM job every day automatically

■ Cost on Railway

Railway gives \$5 free credits per month. A small FastAPI app with SQLite uses roughly \$0.50–1.00/month of compute. You will not exceed the free tier for a college project with 1–2 test households.

09 — Your Demo Script

When you present this to your professor or judges, here is exactly what to show and say. Practice this 3 times before the day.

DEMO 1	Open WhatsApp Show the judge a real phone with WhatsApp open on the test number.
DEMO 2	Trigger the plan manually Hit a <code>/run-now</code> endpoint in your browser (add this to FastAPI for demos). The WhatsApp message arrives on the phone within seconds.
DEMO 3	Read the plan aloud Let the judge read the plan. Point out: 'This used real EskomSePush data for today's schedule and real Open-Meteo solar data.'
DEMO 4	Reply to WhatsApp Type a reply: 'Change washing machine to 4 PM'. Show Gemini parsing it and sending a confirmation back within a few seconds.
DEMO 5	Show the code briefly Show the <code>simulators/inverter.py</code> and <code>agent/scheduler.py</code> files. Explain: 'Hardware is simulated — this class would be replaced by a real inverter API.'
DEMO 6	Show Railway dashboard Show the Railway dashboard with your live deployment and the scheduled 6 AM job.

■ One-line summary for your showcase

"We built an AI energy advisor that pulls real South African load shedding schedules, reasons over them with Gemini, and sends households an optimised daily appliance plan over WhatsApp — completely free to run, no hardware required."